

02285 AI and MAS, F26

Course intro and AI search basics

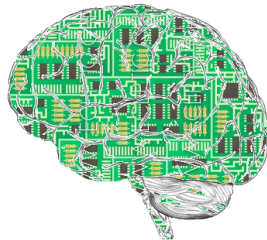
Curriculum for week 1: None, except if you didn't complete the course 02180 Introduction to Artificial Intelligence, then read Chapter 3 of Russell & Norvig, 3ed (old version, relevant chapters available on DTU Learn).

What is AI and MAS?

- **AI (Artificial Intelligence)** is “the science and engineering of making intelligent machines, especially intelligent computer programs” (John McCarthy).
- **Agent:** An autonomous entity which observes and acts upon an environment. It can e.g. be a computer program or a robot, but it can also be a human being. Usually a goal directed entity.
- **MAS (Multi-Agent System):** A system comprised of multiple interacting agents. These agents are usually required to be autonomous (independent) and to form a decentralised system (no controlling agent).



John McCarthy, 2006

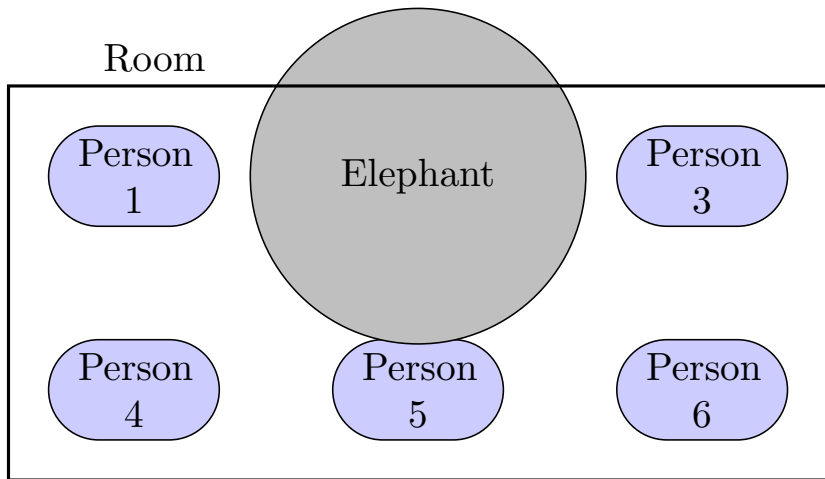


MAS examples

- Groups of hospital robots working in a shared environment.
- Agents for retrieving information on the Web.
- Teams of soccer-playing robots.
- KIVA robots at Amazon.

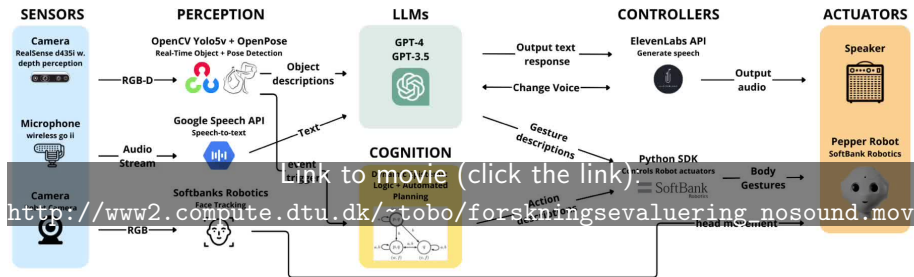


Let's address it...



(poor person 2)

My research, teaching and public outreach using humanoid robots



Research

DTU

"R2DTU
A Pepper robot with social intelligence"

Research on implementing Theory of Mind on robots using dynamic epistemic logic (IJCAI, KR)

Education

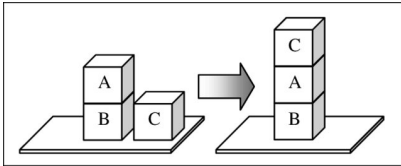


Automated planning, human-robot interact. & plan recognition in course 02182 Symbolic AI

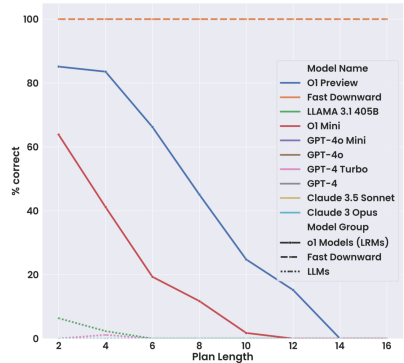
Outreach



LLMs/LRMs performance on search/planning tasks



Plan Length vs Correct Predictions for all Models on Mystery Blocksworld - Plan Generation Zero Shot



Domain	Shots	Instances correct			Average Time Taken (in secs)		
		o1-preview	o1-mini	Fast Downward	o1-preview	o1-mini	Fast Downward
Blocksworld	Zero Shot	587/600 (97.8%)	340/600 (56.6%)	600/600 (100%)	40.43	10.84	0.265
Randomized Mystery Blocksworld	Zero Shot	224/600 (37.3%)	-	600/600 (100%)	111.11	-	0.265

(Valmeekam et al.: LLMs still can't plan; can LRMs? (2004))

General course information

The course consists of two parts of approximately equal length and workload:

- *First half*: **Lectures, exercise classes and 2 mandatory assignments**. Approximately 3.75 ECTS.
- *Second half*: **Mandatory programming project**. Approximately 3.75 ECTS. Implementation (in programming language of your choice) and video hand-in. Concluded by a competition.

MAvis
Level: MAREgExAZ Client: DeePurple State: 0.000 State time:

F	A	A	0										A
F										4		F	F
G	B	B	1										D
G										5		G	G
H	C	C	2										C
H										6		H	H
I	D	D	3										B
I										7		I	I

Link to movie:

http://www2.compute.dtu.dk/~tobo/MAvis/MARegExAZ_02285_level_vert.mov

MAvis
Level: MAREgExAZ Client: DeePurple State: 0.000 State time:

F	A	A	0										A
F										4		F	F
G	B	B	1										D
G										5		G	G
H	C	C	2										C
H										6		H	H
I	D	D	3										B
I										7		I	I

Assessment

The assessment is based exclusively on the written work, that is, the 3 mandatory assignments. These are evaluated as a whole.

- **Warmup Assignment:** Counts approximately 15%.
- **Theory Assignment:** Counts approximately 15%.
- **Programming Project:** Counts approximately 70%.

Course material

- **Main textbook** is *Russell & Norvig: Artificial Intelligence—A Modern Approach*, 4ed.
- **Supplementary chapters on multiagent systems (MAS)** from other books (available on DTU Learn).
- **Supplementary chapters on planning** from *Geffner & Bonet: A Concise Introduction to Models and Methods for Automated Planning*. Available on DTU Learn.

The main course themes are **automated planning** and **multiagent systems**.

The course

The course is designed with an aim to promote and further develop your **independence** and **creativity**. In particular the **programming project** in the second half of the course:

- The project is very open-ended, and you will need to come up with your own unique solution design where it might at first not be clear which parts of the course curriculum are relevant.
- You will likely need to independently search for more literature, e.g. recent research papers, to make a good solution to the project.
- You will need to time manage a big project in a group of 3-5 students, where you are each supposed to spend in total 100+ hours on the project (half of the 7.5 ECTS).

Many students find the programming project quite challenging and too open-ended at first, but most end up being quite happy about it.

Today's subject:

AI search basics

Essentially supposed to be repetition from 02180 Introduction to Artificial Intelligence (Chapter 3 of Russell & Norvig).

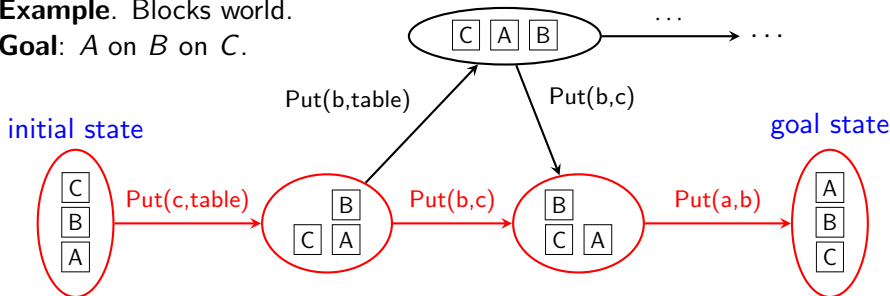
What is a search problem?

Informally: Given 1) **initial state**, 2) **goal states** and 3) **actions**, find a sequence of *actions* that take you from the *initial state* to a *goal state*.

Examples. Find a route from A to B on a map. Organise a dinner party. Charge a satellite. Deliver packages in warehouse.

Example. Blocks world.

Goal: A on B on C.



Solution: Put(c,table), Put(b,c), Put(a,b).

State space of a search problem: The graph of all states reachable from the initial state by using the available actions. E.g. the graph above.

Search problems formally

Formally, a **search problem** has 5 components:

1. s_0 : Initial state
2. $\text{ACTIONS}(s)$: Returns the set of actions **applicable** in state s .
3. $\text{RESULT}(s, a)$: Returns the state s' reached from s by executing the applicable action a .
4. $\text{GOAL-TEST}(s)$: Returns true if s is goal state, otherwise false.
5. $\text{STEP-COST}(s, a)$: The cost of executing action a in s . Most often we will assume $\text{STEP-COST}(s, a) = 1$ for all s and a .

A state g is called a **goal state** if $\text{GOAL-TEST}(g) = \text{true}$.

A **solution** to a search problem is a **sequence of actions** (a **path**) from s_0 to a goal state. It is **optimal** if it has minimum sum of step costs.

The Graph-Search algorithm

Based on Figure 3.7 of Russell & Norvig, 3ed (available on DTU Learn):

function GRAPH-SEARCH (problem) returns a solution, or failure

expandedStates := $\emptyset$

frontier := $\{s_0\}$ (initial state) // we initialise the frontier

loop do

if *frontier* = $\emptyset$ **then return** failure

 choose a state *s* from *frontier*

 remove *s* from *frontier*

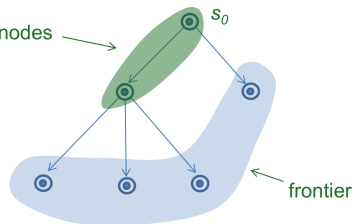
if *s* is a goal state **then return** solution

 add *s* to *expandedStates*

for each child *t* of *s* // we expand *s*

if $t \notin \text{frontier}$ and $t \notin \text{expandedStates}$ **then**

 add child *t* to *frontier*



More precisely and formally:

- “*s* is a goal state”: GOAL-TEST(*s*) is true.
- “child *t* of *s*”: A state $t = \text{RESULT}(s, a)$ for some $a \in \text{ACTIONS}(s)$.

Note: In R&N, 4ed, *reached* denotes $\text{expandedStates} \cup \text{frontier}$.

Uninformed search strategies with Graph-Search

Different search strategies can be achieved by simply changing how **choose state from frontier** and **add child to frontier** work.

Breadth-first search (BFS):

- Frontier is **queue** (FIFO).
- **choose state from frontier**: dequeue state from frontier.
- **add child to frontier**: enqueue state to frontier.

Depth-first search (DFS):

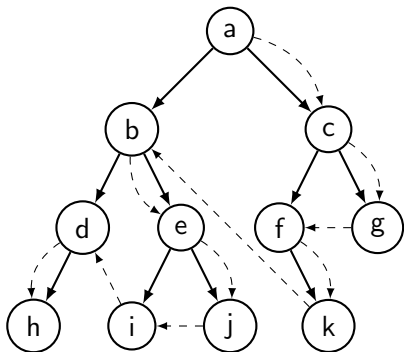
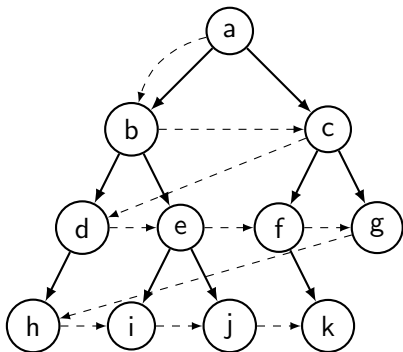
- Frontier is **stack** (LIFO).
- **choose state from frontier**: pop state from frontier.
- **add child to frontier**: push state to frontier.

Order of state expansion: BFS vs DFS

BFS: Breadth-first search.

DFS: Depth-first search.

Order of state expansion:



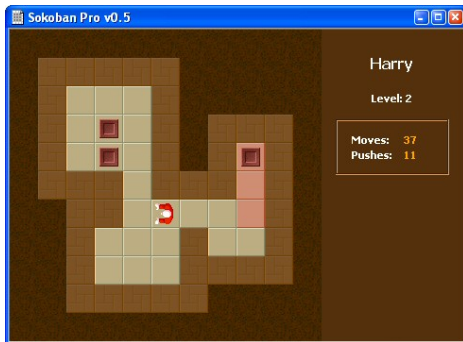
Search problem example: Sokoban

Individually or in pairs: Complete as many levels as possible:

<https://sokoban-game.com/packs/sokogen-990602-levels/>

Note: You can only push boxes straight.

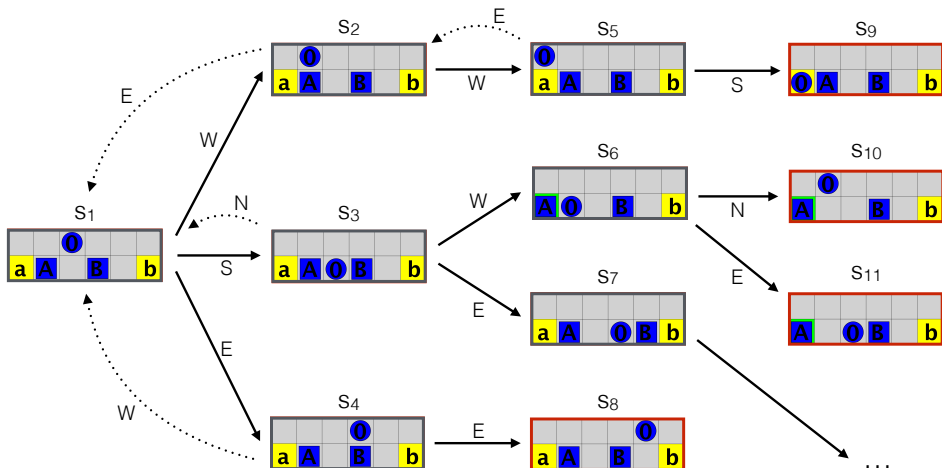
If you are helplessly stuck on a level, you can find the solution on YouTube (search for “Sokogen level n ”).



BFS graph-search example: Sokoban

Children are generated in order *W* (left), *S* (down), *N* (up), *E* (right)

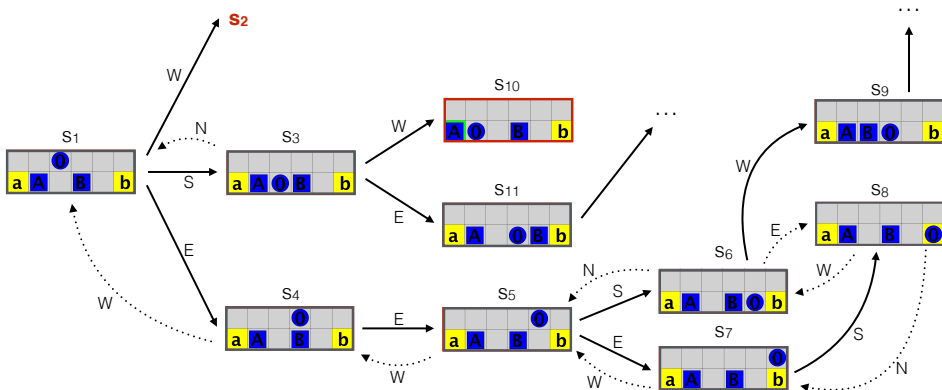
Red border: in frontier. **Grey border:** in expandedStates.



DFS graph-search example: Sokoban

Children are generated in order *W* (left), *S* (down), *N* (up), *E* (right)

Red border: in frontier. **Grey border:** in expandedStates.



Best-first search strategies with Graph-Search

Best-first search:

- Frontier is a **priority queue** where the key of a state s is denoted $f(s)$.
- **choose state s from *frontier***: extract minimal state from priority queue, that is, a state s with minimal $f(s)$.
- **add child t to *frontier***: insert state into priority queue.

Best-first search strategies

- $g(s)$ = the cost of reaching s from the initial state in the search (if the step-cost is 1, it is the number of actions spent to reach s).
- $h(s)$ = a **heuristic value** of s , that is the estimated cost of reaching a goal from s .

The function $h(s)$ is called a **heuristic function** or simply a **heuristic**.

We get different best-first search strategies depending on choice of $f(s)$:

- **Greedy best-first search**: $f(s) = h(s)$.
- **A***: $f(s) = g(s) + h(s)$.
- **WA***: $f(s) = g(s) + W \cdot h(s)$, for some constant $W > 1$.

Which algorithm do we get from letting $f(s) = g(s)$? Dijkstra's. And with step-cost 1 it is simply BFS.

A* is optimal (always finds solution of minimal cost) if either:

- h is *consistent* and if we find a new path to a frontier state with lower g -value, we update the g -value (and parent pointer); or
- h is *admissible* and if we find a new path to a reached state with lower g -value, we put the state on the frontier with the new g -value.

Designing heuristics

Designing a good heuristics $h(s)$ is in many cases a great art form.

What properties should a good heuristics have? Estimate cost/distance to goal as precisely as possible, but still be cheap to compute.

You will work with designing heuristics in the Warmup Assignment.

Example. A simple heuristics for the Sokoban game would be

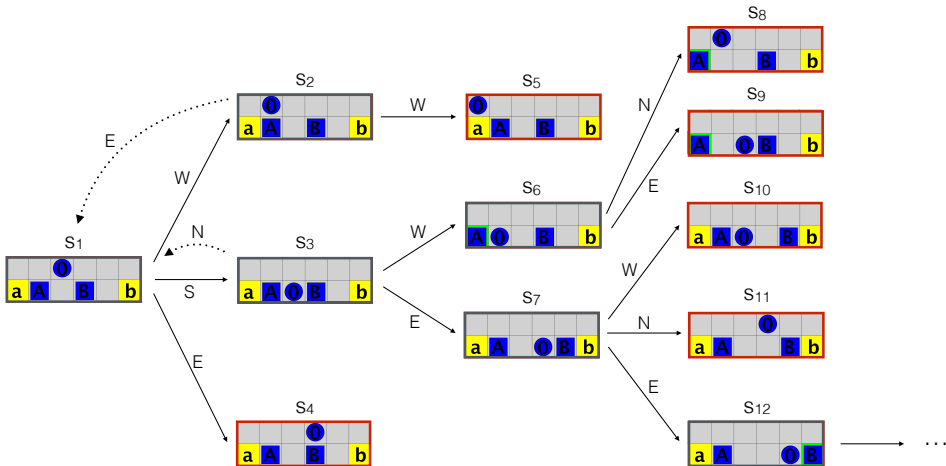
$h(s)$ = the sum of Manhattan distances of boxes to their goals.

This heuristics is used in greedy best-first search on the following page.

Greedy best-first search example

Children are generated in order *W* (left), *S* (down), *N* (up), *E* (right)

Red border: in frontier. **Grey border:** in expandedStates.



Uninformed vs Informed search on Sokoban

Two heuristics:

- **Basic:** $h_{gc}(s)$ is the **goal-count heuristic**, that is, $h_{gc}(s)$ = the number of boxes not in goal cells.
- **A bit more advanced:** $h_w(s)$ sums up Manhattan distances of boxes to goals and returns ∞ when a box is in a corner (deadlock).

Number of generated states for the various search strategies and heuristics (where at least 10,000 states are generated per second):

level	BFS	A^*/h_{gc}	greedy/ h_{gc}	A^*/h_w	greedy/ h_w
Sokogen lvl 2	1,934	1,733	1,575	437	169
Sokogen lvl 3	2,961	2,576	2,953	866	280
Sokogen lvl 14	19,973	17,568	6,346	2,047	798
Sokogen lvl 50	165,389	163,144	125,144	34,452	4,545
Sokogen lvl 76	364,089	347,989	281,507	53,488	31,894
Sokogen lvl 78	613,633	475,604	56,837	88,802	7,327

Relaxed problems and admissible heuristics

Often heuristics are generated via *relaxed problems*.

Relaxed problem: A simplified version of a problem with fewer restrictions. A solution to the original problem should also be a solution to the relaxed problem.

Example. Relaxing the sliding puzzle game:

1. A tile can move to any adjacent square.
2. A tile can move to any square.



Relaxed problems lead to admissible heuristics.

Admissible heuristics. A heuristic is **admissible** if it always returns a value that is less than or equal to the optimal cost of reaching the goal. We often use $h^*(s)$ to denote the optimal cost of reaching the goal from s . A heuristic h is then admissible if $h(s) \leq h^*(s)$ for all s .

Generating heuristics via relaxed problems

Given a problem P , let $h_P^*(s)$ denote the optimal cost to reach a goal from s in problem P . Given any relaxation P' of P , an admissible heuristic h for P can be defined by: $h(s) = h_{P'}^*(s)$.

In words: The **estimated cost** of a solution to the **real problem** is taken to be the **actual cost** of a solution to the **relaxed problem**.

1. Why is h defined above **admissible**, that is, always returns a heuristic value which is less than or equal to the optimal cost? Any solution to P is also a solution to P' . So $h(s) = h_{P'}^*(s) \leq h_P^*(s)$.
2. Which heuristic do we get from the sliding puzzle relaxations of the previous slide (1. Move to any adjacent square; 2. Move to any square)?
1. Sum of Manhattan distances of tiles to goal positions.
2. Number of misplaced tiles (goal count).
3. Does one of the heuristics from the previous question dominate the other? A heuristic h_2 is said to **dominate** a heuristic h_1 if $h_2(s) \geq h_1(s)$ for all s . Yes, the heuristic from 1 dominates the one from 2.
4. Given P' , how do we calculate $h_{P'}^*$? Using breadth-first search on P' .